

CMU 10-703: Deep Reinforcement Learning

Lecture 5: Value-Based Methods in Reinforcement Learning

Max Simchowitz

Carnegie Mellon University

1.

Value-Based Policy Improvement

CMU 10-703: Deep Reinforcement Learning

Policy- and Value-Based Methods

Policy-based

policy $\pi_{\theta}(a \mid s)$

differentiate expected
trajectory return

update policy parameters

Value-based

action value $Q(s, a)$

fit Bellman
consistency

extract greedy policy

Policy-based methods optimize policy parameters directly; value-based methods fit action values and derive the policy from them.

Value Functions

Action value. $Q(s, a)$ measures the expected return after taking action a in state s and then acting optimally.

$$Q(s, a) = r(s, a) + \mathbb{E}_{s' \sim P(\cdot | s, a)} [\gamma V(s')]$$

$$V(s) = \max_a Q(s, a)$$

The action value evaluates one decision; the state value selects the best available decision.

Greedy Policy Extraction

Greedy improvement. Choose an action with the largest estimated advantage or action value in each state.

$$\pi'(s) = \arg \max_a A^\pi(s, a)$$

A value-based policy chooses the action judged best in the current state.

Greedy Policy Extraction

Greedy improvement. Choose an action with the largest estimated advantage or action value in each state.

$$\begin{aligned}\pi'(s) &= \arg \max_a A^\pi(s, a) \\ &= \arg \max_a [Q^\pi(s, a) - V^\pi(s)] = \arg \max_a Q^\pi(s, a).\end{aligned}$$

$V^\pi(s)$ does not depend on the candidate action, so subtracting it cannot change the maximizing action.

Greedy selection from advantage and greedy selection from the corresponding action value produce the same action.

Two Policy-Improvement Mechanisms

Actor-critic

Shift action probabilities through a policy gradient.

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim P^{\pi}(\tau)} \left[\nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t) A^{\pi}(s_t, a_t) \right]$$

Value-based

Replace the policy with the greedy action induced by a fitted value.

$$\pi'(a \mid s) = \mathbf{1} \left\{ a \in \arg \max_{a'} Q^{\pi}(s, a') \right\}$$

Both methods estimate which actions are better, but they encode the improvement step differently.

Actor-critic makes a local parameter update; a value-based method performs explicit greedy policy improvement.

2.

Value Iteration

CMU 10-703: Deep Reinforcement Learning

Bellman Optimality Updates

Evaluate one action, then optimize the next state

$$\begin{aligned}Q_{k+1}(s, a) &= r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} [V_k(s')], \\V_{k+1}(s) &= \max_a Q_{k+1}(s, a).\end{aligned}$$

Each iteration backs up the previous state values through every available action.

Bellman Optimality Updates

Evaluate one action, then optimize the next state

$$\begin{aligned}Q_{k+1}(s, a) &= r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} [V_k(s')] , \\ V_{k+1}(s) &= \max_a Q_{k+1}(s, a).\end{aligned}$$

Combined value update

$$V_{k+1}(s) = \max_a \{ r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} [V_k(s')] \}$$

Substitution yields the Bellman optimality operator on the state-value table.

Value Iteration Algorithm

1. Initialize a value $V_0(s)$ for every state.

Tabular value iteration begins with one scalar estimate per state.

Value Iteration Algorithm

1. Initialize a value $V_0(s)$ for every state.
2. Back up each state-action pair from the previous value table.

$$Q_{k+1}(s, a) = r(s, a) + \gamma \mathbb{E}_{s'}[V_k(s')]$$

The model supplies the expected next-state value for each state-action pair.

Value Iteration Algorithm

1. Initialize a value $V_0(s)$ for every state.
2. Back up each state-action pair from the previous value table.
3. Replace each state value with its best backed-up action value and repeat.

$$Q_{k+1}(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} [V_k(s')]$$

$$V_{k+1}(s) = \max_a Q_{k+1}(s, a)$$

Repeated Bellman backups converge to optimal values in a finite discounted MDP.

Tabular Dynamic Programming

Representation

Store one entry for every state-action pair: a table with $|\mathcal{S}||\mathcal{A}|$ values.

Computation

Each backup requires rewards, transition probabilities, and the current value of successor states.

Discount factor

A smaller γ contracts the update more strongly, but it also places less weight on distant rewards.

Scope

Exact tables become impractical when the state or action space is large or continuous.

Value iteration is exact and convergent in the tabular setting; its table and model requirements prevent direct scaling.

3.

Fitted Q-Iteration

CMU 10-703: Deep Reinforcement Learning

Action-Value Function Approximation

Function approximation. Represent the action-value table by a parameterized function $Q_{\theta}(s, a)$, such as a neural network.



Function approximation shares statistical strength across nearby states and actions instead of allocating an independent table entry to each.

Fitted Q-iteration preserves greedy extraction while replacing exact tabular backups with regression.

Bellman Regression Target

Dynamic-programming relations

$$\begin{aligned}Q(s, a) &= r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)}[V(s')], \\V(s) &= \max_a Q(s, a).\end{aligned}$$

The Bellman relations connect an action value to the best action value after the transition.

Bellman Regression Target

Dynamic-programming relations

$$\begin{aligned}Q(s, a) &= r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)}[V(s')], \\V(s) &= \max_a Q(s, a).\end{aligned}$$

Target induced by the current fit

$$y_k(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} \left[\max_{a'} Q_{\theta_k}(s', a') \right]$$

Fitted Q-iteration turns the Bellman backup from a table operation into a supervised regression target.

Bellman Regression Objective

For sampled transitions $(s, a, r, s') \sim \mathcal{D}$, use the one-sample target from the previous fit:

$$y_k = r + \gamma \max_{a'} Q_{\theta_k}(s', a')$$

$$\mathcal{L}_k(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[(Q_\theta(s, a) - y_k)^2 \right], \quad \theta_{k+1} = \arg \min_{\theta} \mathcal{L}_k(\theta)$$

Each fitted-Q round freezes a Bellman target, then regresses the next action-value function toward that target.

Value Regression and Policy Optimization

Gradient-based regression

Gradients adjust the parameters of the action-value function.

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{L}_k(\theta)$$

Greedy policy improvement

The policy follows from the fitted values without differentiating a policy objective.

$$\pi_{k+1}(s) = \arg \max_a Q_{\theta_{k+1}}(s, a)$$

The optimization uses gradients, but the method is not a policy gradient algorithm.

Fitted Q-iteration differentiates a Bellman regression loss and obtains policy improvement through a separate greedy map.

Fitted Q-Iteration Cycle

collect
transitions

Data

Store (s, a, r, s')
tuples.

A dataset records transitions generated by an initial or current behavior policy.

Fitted Q-Iteration Cycle



Data

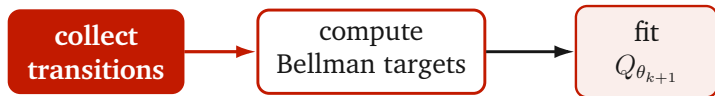
Store (s, a, r, s') tuples.

Backup

Evaluate the next-state maximum.

The current action-value fit supplies a target for every sampled transition.

Fitted Q-Iteration Cycle



Data

Store (s, a, r, s') tuples.

Backup

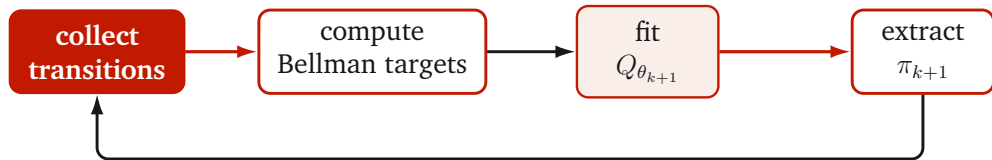
Evaluate the next-state maximum.

Regression

Minimize squared Bellman error.

Regression produces the next approximation to the Bellman backup.

Fitted Q-Iteration Cycle



Data

Store (s, a, r, s') tuples.

Backup

Evaluate the next-state maximum.

Regression

Minimize squared Bellman error.

Improvement

Act greedily under the new fit.

The fitted values define a new greedy policy, which may collect additional experience.

Off-Policy Data Reuse and Coverage

Off-policy reuse

The transition dataset may come from an earlier policy or another behavior policy. Old experience remains available after the greedy policy changes.

Benefit

Reusing transitions can reduce the amount of new environment interaction required per update.

Coverage requirement

The dataset must contain the states and actions whose values affect greedy decisions. Uniform coverage is an ideal, not a realistic default.

Failure mode

A function approximator may assign unsupported values to unseen actions that maximization then selects.

Off-policy learning permits experience reuse, but greedy improvement is only reliable where the dataset supports value comparison.

4.

Deep Q-Networks

CMU 10-703: Deep Reinforcement Learning

Deep Q-Network Update

DQN. Apply fitted Q-iteration with a neural action-value function and stochastic gradient updates.

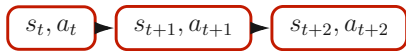
$$\text{TD-Error}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(Q_{\theta}(s, a) - y(s, a) \right)^2 \right]$$

$$\theta_{t+1} = \theta_t - \alpha \nabla_{\theta} \text{TD-Error}(\theta)|_{\theta=\theta_t}$$

DQN performs incremental neural-network regression toward a Bellman target.

Instability in Naive DQN

Correlated updates

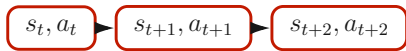


Consecutive transitions resemble one another and overweight recent experience.

Sequential online updates violate the independent-sample picture used by ordinary supervised regression.

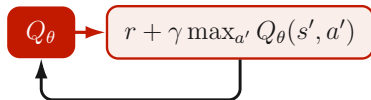
Instability in Naive DQN

Correlated updates



Consecutive transitions resemble one another and overweight recent experience.

Moving targets



The same parameters determine both the prediction and the target being chased.

DQN must control both temporal correlation and feedback between the current prediction and its target.

Experience Replay



Store

Retain transitions across many episodes and policy versions.

Shuffle

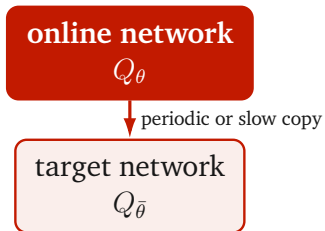
Sample minibatches that mix distant points in time.

Reuse

Train on each environment transition more than once.

Replay converts a correlated transition stream into reusable minibatches drawn from a broader history.

Target Networks



Frozen target parameters

Hold $\bar{\theta}$ fixed while updating the online network.

$$y = r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a')$$

The prediction changes on every optimizer step; the target changes on a slower schedule.

A separate target network slows the feedback loop between the fitted value and the Bellman target.

Hard and Soft Target Updates

Hard update

Keep the target fixed for K optimizer steps, then copy the online parameters.

$$\bar{\theta} \leftarrow \theta \quad \text{every } K \text{ steps}$$

Soft update

Move the target a small fraction toward the online parameters at each update.

$$\bar{\theta} \leftarrow \tau\theta + (1 - \tau)\bar{\theta}, \quad 0 < \tau \ll 1$$

Hard updates create piecewise-constant targets; soft updates create smaller, more frequent target changes.

Both schedules separate the target's time scale from the online network's optimizer steps.

DQN Training Loop

1. Act in the environment and append (s, a, r, s') to the replay buffer.

The behavior policy continually adds new experience to a persistent dataset.

DQN Training Loop

1. Act in the environment and append (s, a, r, s') to the replay buffer.
2. Sample a random minibatch and evaluate fixed-network targets.

$$y_i = r_i + \gamma \max_{a'} Q_{\bar{\theta}}(s'_i, a')$$

Replay sampling and a frozen target network define a more stable regression problem.

DQN Training Loop

1. Act in the environment and append (s, a, r, s') to the replay buffer.
2. Sample a random minibatch and evaluate fixed-network targets.
3. Update the online network, then update the target network on its prescribed schedule.

$$y_i = r_i + \gamma \max_{a'} Q_{\bar{\theta}}(s'_i, a')$$

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} \frac{1}{B} \sum_{i=1}^B (Q_{\theta}(s_i, a_i) - y_i)^2$$

DQN alternates fast online-network updates with slower target-network updates.

Overestimation Bias

Maximization bias. Selecting the largest noisy estimate tends to select a positive estimation error along with a good action.

$$\mathbb{E}\left[\max_a (X_a + \epsilon_a)\right] \geq \max_a \mathbb{E}[X_a] \quad \text{for zero-mean estimation noise } \epsilon_a$$

The maximum of noisy action values is generally larger than the maximum of their means.

Overestimation Bias

Maximization bias. Selecting the largest noisy estimate tends to select a positive estimation error along with a good action.

$$\mathbb{E}\left[\max_a (X_a + \epsilon_a)\right] \geq \max_a \mathbb{E}[X_a] \quad \text{for zero-mean estimation noise } \epsilon_a$$

Value consequence

Bellman targets can become systematically optimistic.

Sampling consequence

Unrepresentative replay batches can reinforce the same estimation errors.

DQN stability depends on both representative replay sampling and control of optimistic max targets.

5.

Evolution Strategies

CMU 10-703: Deep Reinforcement Learning

Black-Box Policy Search

Fitness. Let $F(\theta)$ denote the return obtained by a policy with parameter vector θ .

$$\epsilon \sim \mathcal{N}(0, I_d), \quad \theta = \mu + \sigma\epsilon$$

$$J(\mu) = \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I_d)} [F(\mu + \sigma\epsilon)]$$

Evolution strategies evaluate complete perturbed policies and update the center μ of the search distribution.

Evolution strategies replace action-level or Bellman gradients with black-box comparisons of perturbed policy parameters.

Perturbation Gradient Estimator

Gradient of Gaussian-smoothed fitness

$$\nabla_{\mu} J(\mu) = \frac{1}{\sigma} \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I_d)} [F(\mu + \sigma \epsilon) \epsilon]$$

Fitness weights the random direction used to perturb the current policy parameters.

Perturbation Gradient Estimator

Gradient of Gaussian-smoothed fitness

$$\nabla_{\mu} J(\mu) = \frac{1}{\sigma} \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I_d)} [F(\mu + \sigma \epsilon) \epsilon]$$

Monte Carlo population estimate

$$\widehat{\nabla_{\mu} J} = \frac{1}{n\sigma} \sum_{i=1}^n F(\mu + \sigma \epsilon_i) \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, I_d)$$

A finite population estimates the search gradient by averaging fitness-weighted perturbations.

Evolution Strategy Algorithm

1. Sample independent perturbations around the current mean.

The search distribution generates a population of nearby policy parameters.

Evolution Strategy Algorithm

1. Sample independent perturbations around the current mean.
2. Evaluate the return of every perturbed policy.

$$\epsilon_i \sim \mathcal{N}(0, I_d), \quad F_i = F(\mu_t + \sigma \epsilon_i)$$

Each candidate requires one or more complete environment rollouts to estimate fitness.

Evolution Strategy Algorithm

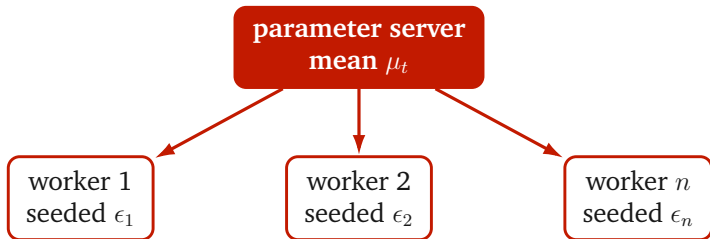
1. Sample independent perturbations around the current mean.
2. Evaluate the return of every perturbed policy.
3. Move the mean toward perturbations with larger returns.

$$\epsilon_i \sim \mathcal{N}(0, I_d), \quad F_i = F(\mu_t + \sigma \epsilon_i)$$

$$\mu_{t+1} = \mu_t + \frac{\alpha}{n\sigma} \sum_{i=1}^n F_i \epsilon_i$$

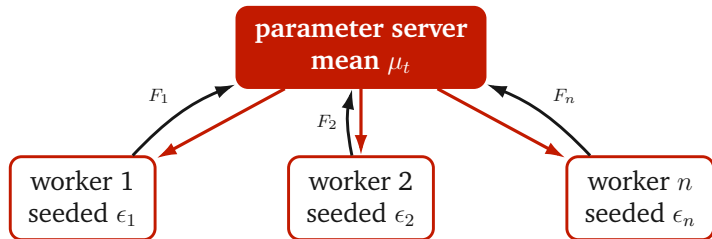
High-return directions receive more weight in the next search mean.

Distributed Evolution Strategies



Workers reconstruct their perturbations from shared random seeds and evaluate candidates independently.

Distributed Evolution Strategies



$$\mu_{t+1} = \mu_t + \frac{\alpha}{n\sigma} \sum_{i=1}^n F_i \epsilon_i$$

Workers communicate scalar returns; the server regenerates perturbations and performs the shared update.

Communication and Optimization Limits

Communication efficiency

Shared seeds let every participant regenerate the same perturbations. Workers send scalar returns rather than full parameter vectors.

Parallel evaluation

Candidate policies can be rolled out independently across workers.

Sample cost

Every search update may require many complete policy evaluations.

Local optima

The search can still concentrate in a poor region. Evaluating candidates across related tasks can reward perturbations that generalize.

Evolution strategies trade environment samples for simple black-box parallelism and low-bandwidth coordination.

6.

Algorithmic Comparison

CMU 10-703: Deep Reinforcement Learning

Value-Based and Evolutionary Methods

Method	Representation	Update	Main constraint
Value iteration	Exact value table	Exact convergent Bellman backup	Small finite state and action spaces
Fitted Q-iteration	Parameterized Q_θ	Batch Bellman regression	Coverage and approximation error
DQN	Neural Q_θ	Replay-based stochastic regression	Moving targets, sampling, and max bias
Evolution strategy	Search distribution over policy parameters	Fitness-weighted perturbations	Large population evaluation cost

Function approximation and black-box search expand scope while weakening guarantees or increasing data demand.

Method Selection Criteria

State and action spaces

Use exact tables only when enumerating state-action pairs is feasible; otherwise use function approximation.

Data reuse

FQI and DQN can train off-policy from stored transitions, subject to coverage.

Stability requirements

DQN requires replay and target-network schedules to control correlated data and moving targets.

Compute and gradient access

Evolution strategies fit settings with abundant parallel rollout capacity or an opaque policy-evaluation pipeline.

Problem scale, available data, stability, and parallel compute determine which optimization burden is acceptable.

Lecture Summary

Bellman improvement

Value iteration alternates action-value backups and greedy maximization.

Neural value learning

FQI and DQN fit Bellman targets; replay and target networks address neural-training instability.

Black-box search

Evolution strategies weight parameter perturbations by complete policy returns.

fit action values $\longrightarrow$ act greedily,
evaluate policy perturbations $\longrightarrow$ move the search mean.

Value-based methods improve a policy through Bellman-consistent action values; evolution strategies improve it through parameter-space fitness.